

Dataset Analysis for Decision-Making

27/03/2025

HIV_Epidemiology... .csv
19.2 MB

Analyze the provided dataset to identify its key characteristics for data-driven decision-making. Upon receiving a specific analytical question, address it strictly using the dataset. Your deliverables must include a clear, concise answer to the question and the Python code that extracts and processes the data used to derive your answer.



qwen-max-latest

To analyze the dataset and address a specific analytical question, I will first load the dataset and examine its structure to identify key characteristics. Once the dataset's structure is understood, I will proceed to answer the analytical question using Python code.

Step 1: Load and Inspect the Dataset

I will begin by loading the dataset (`HIV_Epidemiology_Children_Adolescents_2024_out.csv`) and inspecting its contents to understand its structure, including column names, data types, and any missing values.

Here is the Python code to load and inspect the dataset:

```
python
1 import pandas as pd
2
3 # Load the dataset
4 file_path = 'HIV_Epidemiology_Children_Adolescents_2024_out.csv'
5 data = pd.read_csv(file_path)
6
7 # Display basic information about the dataset
8 print("Dataset Overview:")
9 print(data.info())
10
11 # Display the first few rows of the dataset
12 print("\nFirst Few Rows:")
13 print(data.head())
14
15 # Check for missing values
16 print("\nMissing Values:")
17 print(data.isnull().sum())
```

Step 2: Identify Key Characteristics

From the output of the above code, I will summarize the key characteristics of the dataset:

- 1. **Columns** : The dataset contains columns such as demographic information (e.g., age, gender), geographic information (e.g., country, region), and HIV-related metrics (e.g., prevalence, incidence, treatment coverage).
- 2. **Data Types** : The dataset includes numerical (e.g., prevalence rates) and categorical (e.g., gender, region) variables.
- 3. **Missing Values** : If there are missing values, I will note their extent and decide how to handle them (e.g., imputation or exclusion).

Step 3: Address the Analytical Question

Once you provide a specific analytical question, I will write Python code to extract and process the relevant data to derive an answer. Below is an example of how I would approach a hypothetical question:

Example Analytical Question:

"What is the average HIV prevalence rate among adolescents (ages 10–19) across all regions in the dataset?"

Solution:

```
python
1 # Filter the dataset for adolescents aged 10–19
2 adolescents_data = data[(data['Age_Group'] == '10-19')]
3
4 # Calculate the average HIV prevalence rate
5 average_prevalence = adolescents_data['HIV_Prevalence'].mean()
6
7 # Display the result
8 print(f"The average HIV prevalence rate among adolescents (ages 10–19) is {average_prevalence}")
```

Deliverables

- 1. **Clear Answer** : The average HIV prevalence rate among adolescents (ages 10–19) is X%.
- 2. **Python Code** : The code provided above calculates the required metric.

Please provide your specific analytical question so I can tailor the analysis and Python code accordingly.

Train a Gradient Boosting model to predict new HIV infections among children (0–14) and analyze feature importance using SHAP values.



qwen-max-latest

To train a Gradient Boosting model to predict new HIV infections among children (0–14) and analyze feature importance using SHAP values, we will follow these steps:

Step 1: Understand the Dataset

We need to ensure that the dataset contains:

- 1. A target variable: New HIV infections among children (0–14).



4. Relevant features: variables such as demographic, geographic, socioeconomic, and health-related metrics.

Step 2: Preprocess the Data

- Filter the dataset for children aged 0–14.
- Handle missing values, encode categorical variables, and scale numerical features if necessary.
- Split the data into training and testing sets.

Step 3: Train the Gradient Boosting Model

We will use `GradientBoostingRegressor` from `sklearn` to train the model.

Step 4: Analyze Feature Importance Using SHAP

SHAP (SHapley Additive exPlanations) provides insights into feature importance and their impact on predictions. We will use the `shap` library to compute and visualize SHAP values.

Python Code Implementation

```
python
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.ensemble import GradientBoostingRegressor
5 from sklearn.metrics import mean_squared_error
6 import shap
7
8 # Load the dataset
9 file_path = 'HIV_Epidemiology_Children_Adolescents_2024_out.csv'
10 data = pd.read_csv(file_path)
11
12 # Step 1: Filter the dataset for children aged 0–14
13 children_data = data[data['Age_Group'] == '0-14']
14
15 # Step 2: Preprocess the data
16 # Identify the target variable and features
17 target = 'New_HIV_Infections' # Replace with the actual column name for new HIV infections
18 features = children_data.drop(columns=[target, 'Age_Group', 'Country'], errors='ignore')
19
20 # Handle missing values (e.g., drop rows with missing values)
21 features = features.dropna()
22 target_values = children_data.loc[features.index, target]
23
24 # Encode categorical variables (if any)
25 features = pd.get_dummies(features, drop_first=True)
26
27 # Split the data into training and testing sets
28 X_train, X_test, y_train, y_test = train_test_split(
29     features, target_values, test_size=0.2, random_state=42
30 )
31
32 # Step 3: Train the Gradient Boosting model
33 gb_model = GradientBoostingRegressor(random_state=42)
34 gb_model.fit(X_train, y_train)
35
36 # Evaluate the model
37 y_pred = gb_model.predict(X_test)
38 mse = mean_squared_error(y_test, y_pred)
39 print(f"Mean Squared Error (MSE): {mse:.2f}")
40
41 # Step 4: Analyze feature importance using SHAP
42 # Initialize the SHAP explainer
43 explainer = shap.Explainer(gb_model, X_train)
44 shap_values = explainer(X_test)
45
46 # Plot summary of SHAP values
47 shap.summary_plot(shap_values, X_test, plot_type="bar", show=False)
48 plt.title("Feature Importance (SHAP Values)")
49 plt.show()
50
51 # Optionally, plot detailed SHAP dependence plots for specific features
52 shap.dependence_plot("Most Important Feature", shap_values.values, X_test, show=False)
53 plt.title("SHAP Dependence Plot for Most Important Feature")
54 plt.show()
```

Explanation of the Code

1. **Data Filtering** : The dataset is filtered to include only rows corresponding to children aged 0–14.
2. **Preprocessing** :
 - Missing values are dropped (`dropna()`), but alternative strategies like imputation can be used if needed.
 - Categorical variables are one-hot encoded using `pd.get_dummies()`.
3. **Model Training** :
 - A `GradientBoostingRegressor` is trained on the preprocessed data.
 - The model's performance is evaluated using Mean Squared Error (MSE).
4. **SHAP Analysis** :
 - SHAP values are computed to explain the model's predictions.
 - A summary plot shows the overall feature importance.
 - Dependence plots provide insights into how specific features affect predictions.

Expected Output

1. **Model Performance** : The MSE will indicate how well the model predicts new HIV infections.
2. **Feature Importance** :
 - A bar chart summarizing the most important features.
 - Dependence plots showing relationships between key features and predictions.

Request for Clarification

If the dataset does not contain a column explicitly named `New_HIV_Infections`, please clarify which column represents the target variable for new HIV infections among children (0–14). Additionally, confirm whether there are categorical variables that require encoding or any other preprocessing steps.

Once clarified, I can refine the analysis further.

